Section 1 · Where am I?

- **`pwd`** — "print working directory." Prints the full path of the folder your terminal is currently "standing in." Everything you do (create files, run scripts) happens relative to this location unless you say otherwise.
- **`ls`** — Lists the files and folders in your current directory (visible ones only).
- **`ls -a`** — Same as `ls`, but the `-a` flag means "all," including hidden files/folders (anything starting with a `.`, like `.bashrc` or `.venv`). Unix hides dotfiles by default because they're usually config/system files you don't need to see day-to-day.
- **`ls -l`** — "Long" format. Instead of just names, it shows a detailed table: permissions, number of links, owner, group, file size, and last-modified date. The very first column (e.g. `-rwxr-xr-x`) encodes the file type (`-` = regular file, `d` = directory) and permissions for owner/group/others.

Section 2 · Build the folder

- **`mkdir coding-cafe`** — "make directory." Creates a new folder.
- **`cd coding-cafe`** — "change directory." Moves your terminal's working location into that folder.
- **`touch README.md`** — Creates an empty file if it doesn't exist (or just updates its timestamp if it does).
- **`ls coding-cafe/week02`** run from outside the folder — this works because `coding-cafe/week02` is a **relative path**: it's interpreted relative to wherever you currently are. Since `coding-cafe` is a subfolder of your current location, the shell can find it without you `cd`-ing there. An **absolute path** would start from the filesystem root, e.g. `/home/yourname/coding-cafe/week02`.
- **`cd ~`** — `~` is shorthand for your home directory, so this jumps straight there regardless of where you were.
- **`ls -R coding-cafe`** — `-R` means recursive: it lists the contents of `coding-cafe` *and* every subfolder inside it, showing the whole tree.

Section 3 · Move, copy, rename, delete

- **`cp README.md README-backup.md`** — "copy." Duplicates the file under a new name; the original stays put.
- **`mv README-backup.md week01/`** — "move." Since the destination is a folder, this relocates the file into `week01/` without changing its name.
- **`cd week01`** — moves into that folder.
- **`mv README-backup.md old-readme.md`** — this time the destination isn't a folder, it's a new filename in the *same* location — so `mv` renames the file instead of moving it. This is the "two different jobs" the question is pointing at: `mv` = move **and** rename, depending on what you give it as the target.

- **`rm old-readme.md`** — "remove." Permanently deletes the file. There's no recycle bin in the terminal — once it's gone, it's gone (no `-r` was used here since it's just a single file, not a folder).

Section 4 · PATH and command lookup

- **`echo $PATH`** — `echo` prints text; `$PATH` is an environment variable holding a colon-separated list of folders. When you type a command name (like `ls`), the shell searches these folders in order and runs the first matching executable it finds.
- **`which python3`**, **`which pip`**, **`which ls`** — `which` tells you the *exact* file path of the program that would run for that command name, by searching `$PATH` the same way the shell does.
- **`pyton3`** (typo) — the shell searches every folder in `$PATH` for a program named `pyton3`, finds nothing, and reports `command not found`. This is a PATH lookup failure, not a "python is broken" error — the shell literally couldn't find anything by that name anywhere it knows to look.

Section 5 · Virtual environments (venv)

- **`python3 -m venv .venv`** — runs the `venv` module (`-m` = run a module as a script) built into Python, which creates a new folder called `.venv` containing a self-contained copy of Python and its own isolated package storage. This is your "project's own box."
- **`ls`** / **`ls -a`** — plain `ls` won't show `.venv` because it starts with a dot (hidden), same rule as Section 1.
- **`source .venv/bin/activate`** — `source` runs a script *in your current shell* (rather than a subprocess), so its changes to environment variables persist. `activate` rewrites your `$PATH` so it points `python3` and `pip` at the versions inside `.venv` first, and it also changes your prompt to show `(.venv)` as a reminder you're "inside the box."
- **`which python3`** (after activating) — now shows a path *inside* `.venv/bin/python3` rather than the system Python, proving `activate` really did change your `PATH` search order.
- **`pip list`** — shows every package currently installed in the active Python environment.
- **`pip install requests`** — downloads and installs the `requests` package, but only inside the currently active venv — it won't touch your system-wide Python.
- **`pip list`** (again) — now includes `requests` (and its dependencies), showing the install took effect only in this isolated environment.
- **`deactivate`** — a function that `activate` defined; it undoes everything, restoring your original `$PATH` and prompt.
- **`which python3`** (after deactivate) — now points back to the system Python, since the venv's folder has been removed from `$PATH`.

Section 6 · Manual pages

- **`man ls`** — opens the manual page for `ls`, a built-in reference describing every flag/option it supports. `q` quits the viewer (called `less` under the hood) and returns you to the shell.
- **`man man`** — the manual entry for the `man` command itself, describing how manual pages work.

Section 7 · Working faster

- **Tab completion** (`cd cod` + Tab) — the shell auto-completes the rest of the name if there's only one match; pressing Tab twice when it's ambiguous (multiple folders start with `cod`) lists all the possibilities so you can keep typing to disambiguate.
- **Up arrow** — cycles backward through your previously typed commands (shell history), so you can reuse or edit them instead of retyping.
- **`history`** — prints your entire command history as a numbered list.
- **`python3`** (run alone) — with no script argument, this launches the interactive Python REPL (read-eval-print loop), dropping you into a `>>>` prompt inside Python itself, not the shell.
- **Ctrl+C** — sends an interrupt signal to the currently running foreground program, asking it to stop immediately. In the Python REPL, it typically just cancels the current input line/expression rather than closing Python entirely (you'd see `KeyboardInterrupt`).
- **Ctrl+D** — sends an "end of input" (EOF) signal. In the Python REPL, this tells Python there's no more input coming, which closes it gracefully — equivalent to typing `exit()`.

Section 8 · Logging your work

- **`history > week02/lab01-log.txt`** — `history` prints your command list to the screen as usual, but `>` is the **redirection operator**: instead of letting that output go to your terminal, it sends it into the file `lab01-log.txt` (creating the file, or overwriting it if it already existed). This is how you turn terminal output into a saved file.

Bonus ("if you finished early")

- **`ls -lh`** — like `ls -l`, but `-h` makes file sizes "human-readable" (e.g. `4.0K`, `1.2M` instead of raw byte counts).
- **`cd -`** — jumps to whatever directory you were in *before* your last `cd`, letting you toggle back and forth between two locations.
- **`mkdir -p a/b/c/d`** — `-p` means "create parent directories as needed." Without it, `mkdir` would fail if `a` or `a/b` didn't already exist; `-p` builds the whole nested chain in one shot.

- **`pip freeze`** — like `pip list`, but outputs packages in a machine-readable `name==version` format suitable for pasting into a `requirements.txt` file, so someone else can install the exact same setup.
- **`man grep`** — just previewing that `grep` (a tool for searching text inside files) exists; no need to learn it yet.